

Experiment No.	09
Experiment Title	Perceptron vs Multilayer Perceptron (A/B Experiment) with Hyperparameter Tuning
GitHub Repository:	https://github.com/KJayasuriya/ml_laboratory
Student Name	K. Jayasuriya
Register Number	3122247001026
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	September 19, 2026

1 Objective

The objective of this experiment is to implement and compare a Single-Layer Perceptron Learning Algorithm (PLA) with a Multilayer Perceptron (MLP).

The experiment focuses on:

- Implementing PLA from scratch.
- Implementing an MLP for multi-class classification.
- Studying the effect of activation functions, optimizers, learning rates and batch sizes.
- Selecting a suitable MLP configuration through hyperparameter tuning.
- Comparing PLA and MLP using standard evaluation metrics.
- Studying training convergence and model performance.

2 Problem Statement

The task is to classify handwritten English characters using two different neural network approaches.

Model A is a Single-Layer Perceptron Learning Algorithm (PLA), which uses a step activation function and a weight update rule.

Model B is a Multilayer Perceptron (MLP), which contains hidden layers and nonlinear activation functions. The MLP is trained using backpropagation.

The performance of both models is compared using accuracy, precision, recall, F1-score, confusion matrices and ROC curves.

3 Dataset Description

The dataset used in this experiment is the English Handwritten Characters Dataset.

Dataset Name	English Handwritten Characters Dataset
Dataset Source	English Handwritten Characters Dataset
Number of Images	3410
Number of Classes	62
Classes	0–9, A–Z and a–z

Image Type	Handwritten character images
Image Size Used	28×28 pixels
Features After Flattening	784
Train-Test Split	80% training and 20% testing

The dataset contains handwritten characters belonging to 62 different classes. Each image was converted to grayscale and resized before being used for model training.

4 Data Preprocessing

The following preprocessing steps were performed:

1. The handwritten character images were loaded from the dataset.
2. Images were converted to grayscale.
3. Each image was resized to 28×28 pixels.
4. The images were flattened into 784 features.
5. Pixel values were normalized to the range 0 to 1.
6. Character labels were encoded into numerical class labels.
7. The dataset was divided into training and testing sets using an 80:20 stratified split.

After preprocessing, the feature matrix contained 3410 samples and 784 features.

5 Perceptron Learning Algorithm

The Perceptron is a single-layer model that uses a step activation function. It updates its weights whenever the prediction differs from the target.

The weight update rule used in the experiment is:

$$w_{t+1} = w_t + \eta(y - \hat{y})x$$

where:

- w is the weight vector.
- η is the learning rate.
- y is the actual target.
- $\hat{y}$ is the predicted value.
- x is the input vector.

Since the dataset contains 62 classes, a separate binary perceptron was trained for each class using a one-vs-rest approach.

5.1 PLA Results

The PLA was trained for 20 epochs with a learning rate of 0.01.

Table 2: PLA Performance

Metric	Score
Accuracy	0.1114
Precision	0.2210
Recall	0.1114
F1-Score	0.0869
ROC-AUC	0.7517

5.2 PLA Confusion Matrix

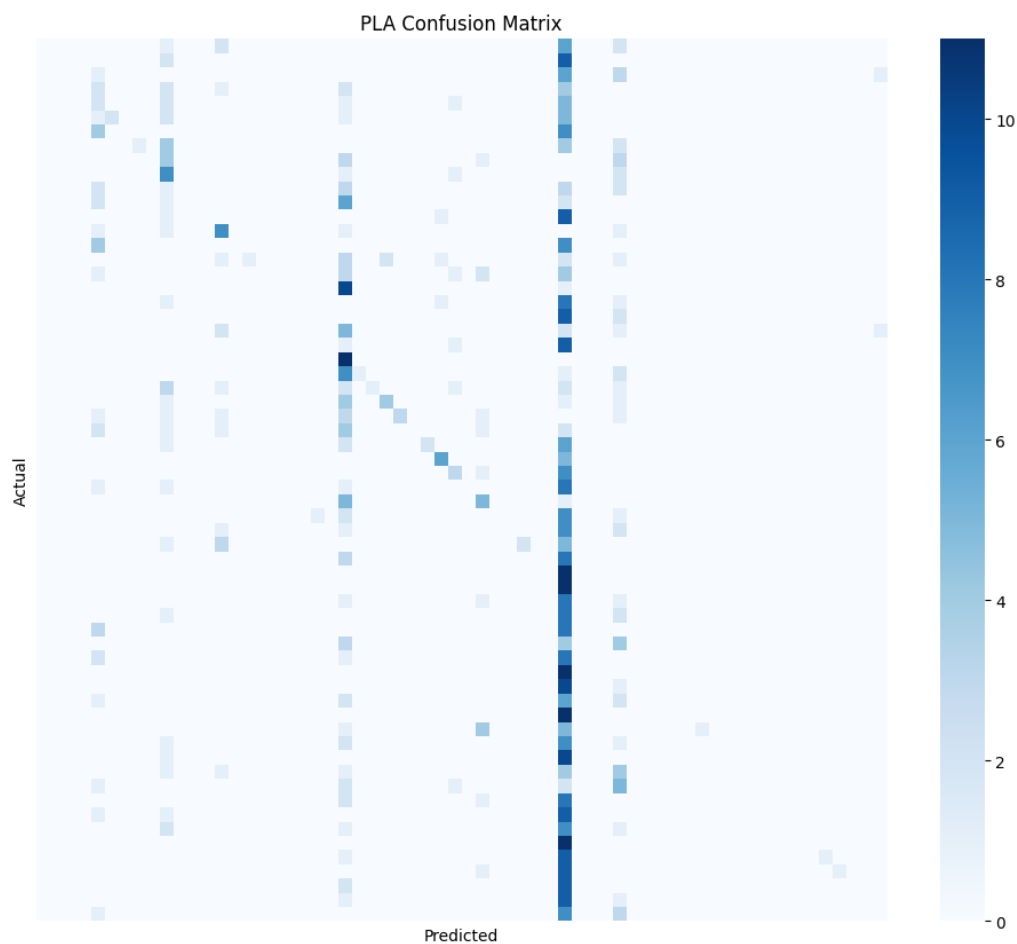


Figure 1: PLA Confusion Matrix

5.3 PLA Convergence

The training error was recorded for each epoch to observe the convergence behaviour of the perceptron.

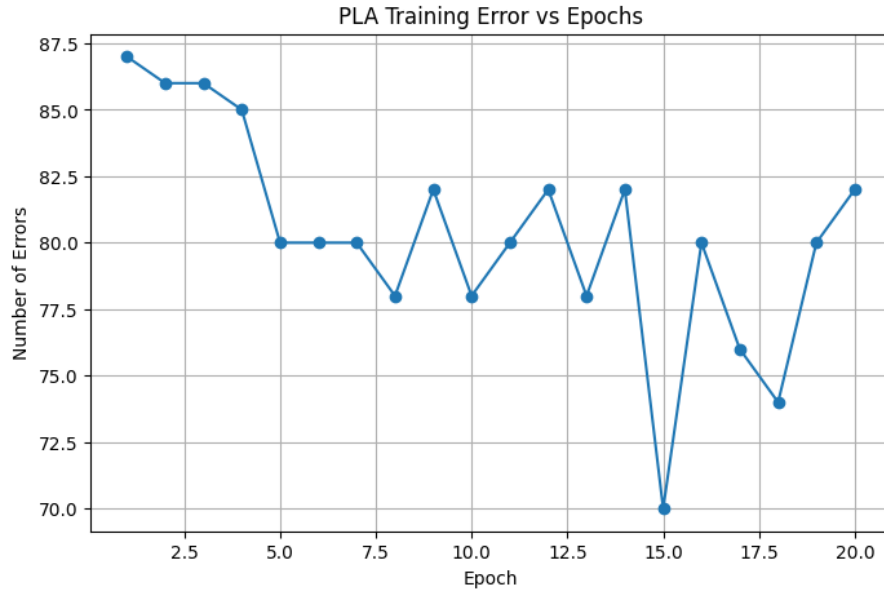


Figure 2: PLA Training Error vs Epochs

6 Multilayer Perceptron

The Multilayer Perceptron contains an input layer, hidden layer and output layer.

The basic architecture used in this experiment was:

Input Layer $\rightarrow$ Hidden Layer $\rightarrow$ Output Layer

The hidden layer contained 128 neurons.

The MLP was trained using backpropagation. ReLU and Tanh activation functions were considered during hyperparameter tuning.

Adam and SGD were also tested as optimizers.

7 Hyperparameter Tuning

Multiple MLP configurations were tested by changing the activation function, optimizer, learning rate and batch size.

Table 3: MLP Hyperparameter Tuning Results

Activation	Optimizer	Learning Rate	Batch Size	Accuracy	F1-Score
ReLU	Adam	0.001	32	0.0161	0.0005
ReLU	Adam	0.001	64	0.0880	0.0560
ReLU	SGD	0.010	32	0.0161	0.0005
ReLU	SGD	0.010	64	0.1862	0.1561
Tanh	Adam	0.001	32	0.1891	0.1492
Tanh	Adam	0.001	64	0.2786	0.2534

The configuration with the highest F1-score was selected as the final MLP configuration.

7.1 Selected Hyperparameters

Table 4: Final MLP Hyperparameters

Parameter	Selected Value
Hidden Layer	128 neurons
Activation Function	Tanh
Optimizer	Adam
Learning Rate	0.001
Batch Size	64
Maximum Epochs	150

The selected configuration was chosen based on its F1-score on the test data among the configurations evaluated during tuning.

8 MLP Results

The final MLP was trained using the selected hyperparameters.

Table 5: MLP Performance

Metric	Score
Accuracy	0.4238
Precision	0.4514
Recall	0.4238
F1-Score	0.4133
ROC-AUC	0.9237

8.1 MLP Confusion Matrix

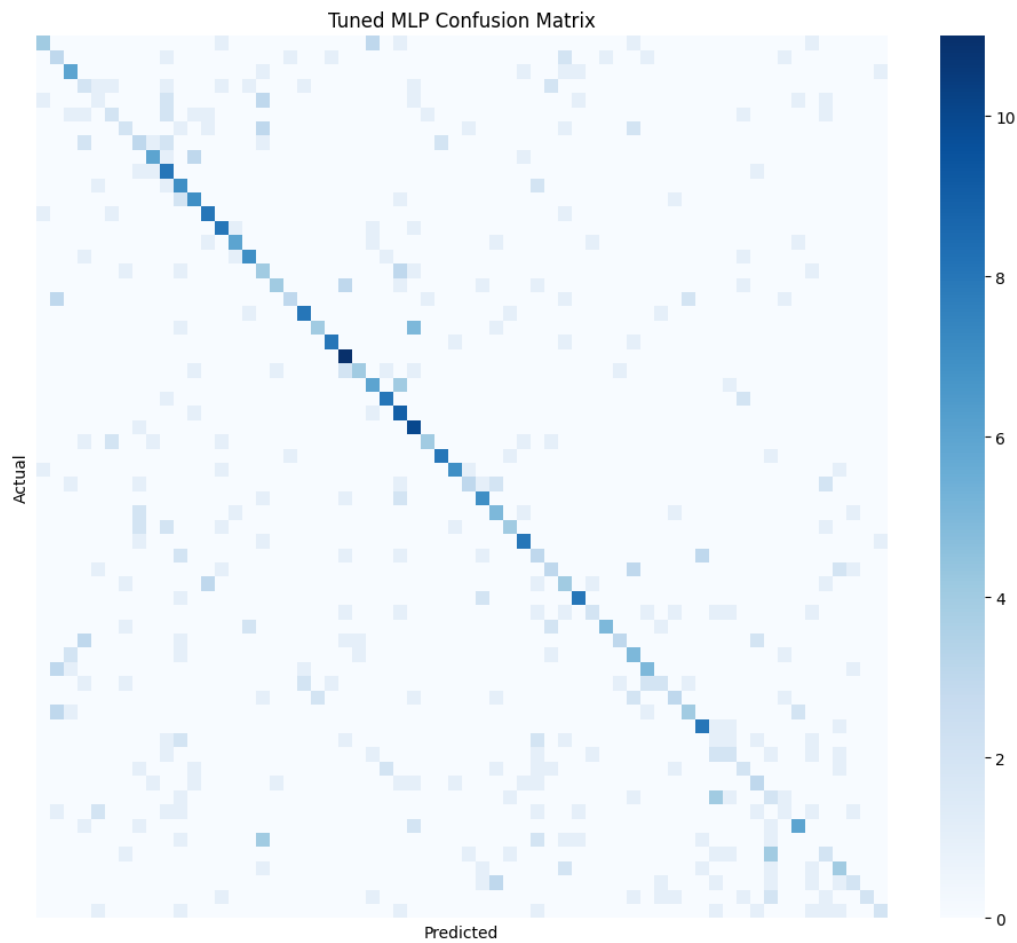


Figure 3: Tuned MLP Confusion Matrix

8.2 MLP Convergence

The training loss decreased as the number of epochs increased. This shows the learning behaviour of the MLP during training.

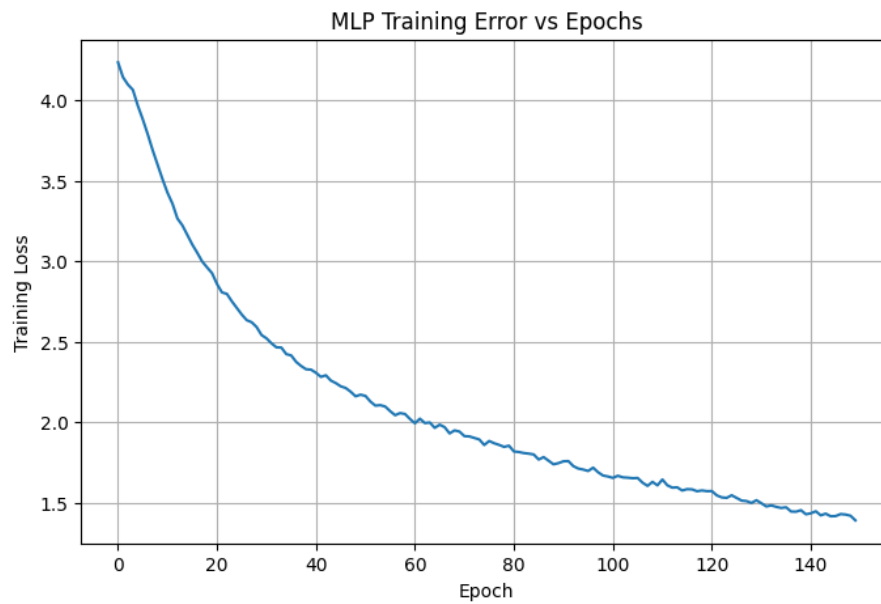


Figure 4: MLP Training Error vs Epochs

9 A/B Comparison

The performance of PLA and the tuned MLP was compared using the same test set.

Table 6: PLA vs MLP Performance Comparison

Metric	PLA	MLP
Accuracy	0.1114	0.4238
Precision	0.2210	0.4514
Recall	0.1114	0.4238
F1-Score	0.0869	0.4133
ROC-AUC	0.7517	0.9237

9.1 Performance Comparison

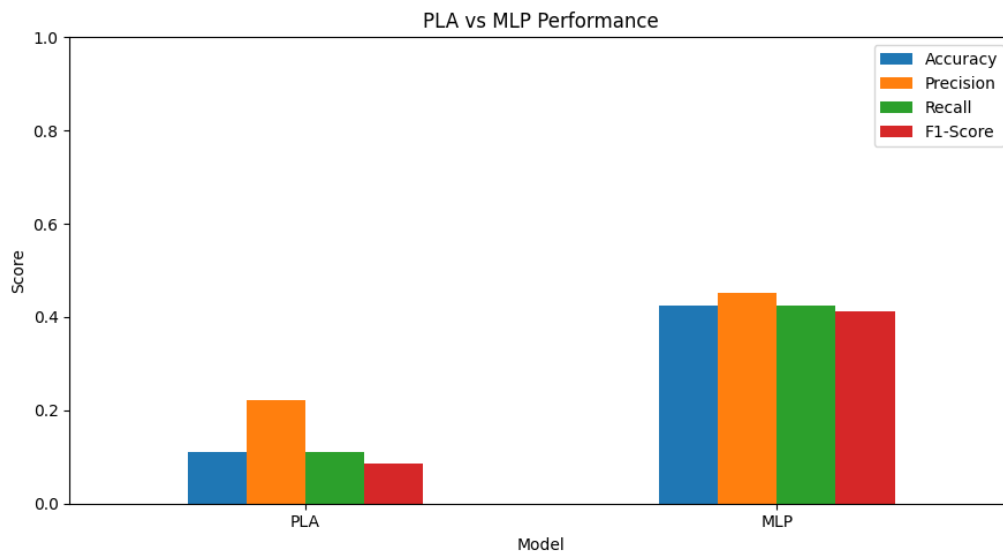


Figure 5: Performance Comparison between PLA and MLP

9.2 ROC Curve

The ROC curves of the two models were compared using a micro-average approach.

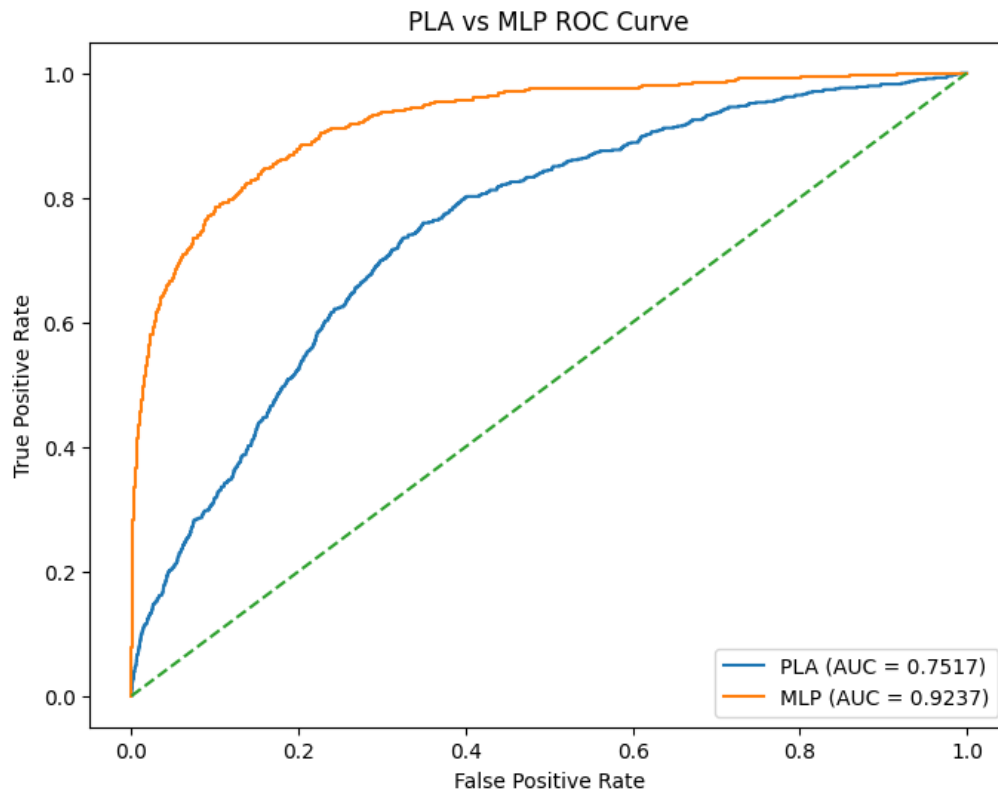


Figure 6: ROC Curves of PLA and MLP

10 Discussion

The PLA and MLP showed different performance characteristics on the handwritten character classification task.

The PLA uses a single layer and a step activation function. Therefore, its ability to learn complex decision boundaries is limited. The dataset contains 62 handwritten character classes, and the classes can have overlapping visual patterns. This makes the classification problem more difficult for a single-layer perceptron.

The MLP contains a hidden layer and uses a nonlinear activation function. This allows it to learn more complex relationships between the image features and character classes.

Hyperparameter tuning also affected the MLP performance. Different learning rates, batch sizes, activation functions and optimizers produced different results.

The selected configuration was the one that produced the highest F1-score among the tested configurations.

11 Observation Questions

11.1 1. Why does PLA underperform compared to MLP?

PLA has only one layer and uses a step function, so it cannot learn complex patterns well. MLP has hidden layers and nonlinear activation, so it can handle the handwritten character data better.

11.2 2. Which hyperparameters had the most impact on MLP performance?

The activation function, optimizer and batch size affected the results. Among the tested configurations, Tanh with Adam and batch size 64 gave the highest F1-score.

11.3 3. Did optimizer choice affect convergence?

Yes. Adam and SGD produced different training behaviour and performance. Adam gave better results in the tested configurations.

11.4 4. Did adding more hidden layers always improve results?

No. More layers do not always mean better performance. A deeper model can also take more time to train and may overfit.

11.5 5. Did the MLP show overfitting? How could it be mitigated?

The training loss kept decreasing, but this alone is not enough to confirm overfitting. It can be reduced using early stopping, dropout, regularization or a simpler model.

12 Conclusion

The experiment compared a Single-Layer Perceptron Learning Algorithm with a Multi-layer Perceptron for handwritten character classification.

PLA was implemented from scratch using a step activation function and the perceptron weight update rule. The MLP was implemented with a hidden layer and nonlinear activation functions.

Hyperparameter tuning was performed by testing different activation functions, optimizers, learning rates and batch sizes. The best configuration was selected based on the observed F1-score.

The comparison shows the difference between a simple single-layer model and a multilayer neural network on a multi-class image classification problem. The convergence curves and evaluation metrics were also used to study the training behaviour and final performance of both models.

13 References

- Scikit-learn: Neural Network Models.
- Scikit-learn: MLPClassifier Documentation.
- Scikit-learn: Classification Metrics.
- English Handwritten Characters Dataset.